

D4: Greedy Algorithms and More DP

**STOP DOING
ALGORITHM ANALYSIS!**

- PIECES OF LOGIC were not supposed to be analysed
- YEARS of ANALYSING COMPLICATED ALGORITHMS yet NO REAL WORLD USE FOUND for using anything **beyond THE GREEDY ALGORITHM**
- Wanted to analyse an algorithm for a laugh? We had a tool for that it's called **/usr/bin/time**.
- "Please give me $O(n^3)$ apples." "Okay, that will require $\Theta(n^{69} \log n)$ time and $\Omega(n \log^* n / e)$ space please." - Statements dreamed up by the **UTTERLY DERANGED**

LOOK what COMPUTER SCIENTISTS have been doing with all the MATHS and COMPUTERS we have given to them

(these are real algorithms, analysed by real computer scientists)



??????



??????????



????????????????

They played us for **ABSOLUTE FOOLS!**

Sec 16: Fr 12:30pm-1:30pm FXB 1024
IA: Nick Frierler

Agenda

- ▶ DP dimensionality
- ▶ Shortest path DP
 - ▶ SSSP (Bellman-ford)
 - ▶ APSP (floyd-Warshall)
- ▶ Greedy algorithms
 - ▶ Exchange arguments

DP Dimensionality

More dimensions!?

- ▶ A lot of the DP examples we have seen so far have been 1-D DP problems
 - ▶ Fibonacci
 - ▶ LIS (using END-LIS as an auxiliary DP table)
 - ▶ Merge-Sort
- ▶ But wait, there's more!
- ▶ Some problems are more naturally suited for multi-dimensional DP tables
 - ▶ The SugarRush DP table from hw 3
 - ▶ Contiguous subarrays of an array

Shortest Path DP

Setup for shortest path problems

- ▶ Weighted directed graph $G = (V, E)$ with a weighted function $w: E \rightarrow \mathbf{R}$ that maps each edge to a weight/cost/length
- ▶ Individual edges can have negative weight
- ▶ No negative-weight cycles
 - ▶ Otherwise the (minimum) distance between some vertices of the graph will not be well-defined

Key observations

- ▶ If P is a shortest path from s to t that goes through vertex v , then P must be:
 - ▶ A shortest path from s to v , followed by
 - ▶ A shortest path from v to t
 - ▶ (This is a good exercise for proof by contradiction)
- ▶ Because there are no negative-weight cycles in the graph, if there is a shortest path from s to t , then there is a shortest path from s to t with no cycles
 - ▶ We call this a simple path because no vertex is visited more than once

Single-source shortest path (SSSP)

- ▶ Given a source vertex s , find the shortest path from s to every other vertex
 - ▶ Value version: find the lengths of these shortest paths
- ▶ Bellman-Ford
 - ▶ $d^{(i)}(s,t)$ is the length of a shortest path from s to t that uses exactly i edges
 - ▶ $d(s,t) = \min\{ d^{(1)}(s,t), \dots, d^{(n-1)}(s,t) \}$
 - ▶ To calculate $d^{(i)}(s,t)$ we consider $d^{(i-1)}(s,u)$ for all vertices u such that (u,t) is an edge in the graph and add the weight of edge (u,t)

All-pairs shortest paths (APSP)

- ▶ For every pair of vertices (s, t) , find the shortest path from s to t
 - ▶ Value version: find the lengths of these paths
- ▶ Floyd-Warshall
 - ▶ Let $v_1, \dots, v_n$ be a fixed ordering of the vertices of the graph
 - ▶ $d^{[i]}(s, t)$ is the length of a shortest path from s to t whose internal vertices (if any) are in $\{v_1, \dots, v_i\}$
 - ▶ $d(s, t) = d^{[n]}(s, t)$
 - ▶ To calculate $d^{[i]}(s, t)$ we take the minimum of $d^{[i-1]}(s, t)$ and $d^{[i-1]}(s, v_i) + d^{[i-1]}(v_i, t)$

Greedy Algorithms

Gimme gimme gimme

- ▶ A *greedy algorithm* computes a solution to an optimization problem by making a sequence of locally optimal choices
 - ▶ Cannot “change its mind” about a previously-made decision
- ▶ In general, no guarantee that a greedy solution is optimal (or even good)
 - ▶ However, there are scenarios where they are (and we can prove they are optimal!)

Exchange argument recipe

- ▶ Let $\mathbf{ALG} = \{s_1, s_2, \dots, s_m\}$ denote the (greedy) algorithm's final output
 - ▶ Think of these as the *choices* that the algorithm (greedily) makes
- ▶ Let $\mathbf{OPT}$ be some *arbitrary* optimal solution
- ▶ If $\mathbf{ALG} = \mathbf{OPT}$, then we're done
- ▶ Otherwise $\mathbf{ALG} \neq \mathbf{OPT}$, so there's some element in $\mathbf{ALG}$ that's not in $\mathbf{OPT}$, call this s_{diff}
- ▶ Modify $\mathbf{OPT}$ so that it remains optimal but contains s_{diff} to get a new optimal solution $\mathbf{OPT}'$
- ▶ Apply induction on the number of elements in the output where $\mathbf{ALG}$ and *some* optimal solution differ to conclude that there exists *some* optimal solution that matches $\mathbf{ALG}$ fully
 - ▶ This means $\mathbf{ALG}$ is optimal!

Any questions before
discussion problems?

Discussion Problems

Q1: Longest Palindromic Subsequence

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness. Given a string S , determine the length of a longest subsequence of S (the characters do not need to be consecutive) that is a palindrome. Recall that a palindrome is a string which is the same forwards and backwards (e.g., “racecar”).

Q1: Longest Palindromic Subsequence

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness. Given a string S , determine the length of a longest subsequence of S (the characters do not need to be consecutive) that is a palindrome. Recall that a palindrome is a string which is the same forwards and backwards (e.g., “racecar”).

$$\text{LPS}[i, j] = \begin{cases} 2 + \text{LPS}[i + 1, j - 1] & \text{if } S[i] = S[j] \\ \max\{\text{LPS}[i + 1, j], \text{LPS}[i, j - 1]\} & \text{otherwise.} \end{cases}$$

Attendance



Q2: Bellman-Ford and Floyd-Warshall

- If p is the *unique* shortest s -to- t path in a graph—i.e., every other s -to- t path has strictly larger length—and v is a vertex on p , then the s -to- v portion of p is the *unique* shortest s -to- v path in the graph.
- In the Floyd–Warshall recurrence, the values $d^{[i]}(s, t)$ are *non-increasing* as i increases, i.e., $d^{[j]}(s, t) \leq d^{[i]}(s, t)$ for all $j \geq i$ and all vertices s, t .
- If a graph has no negative-weight cycle, then no shortest path in the graph contains a cycle.
- To fill each individual entry of the DP table in the Floyd–Warshall algorithm, we need to look at only a constant number of cells, whereas this may not be the case in the Bellman–Ford algorithm.

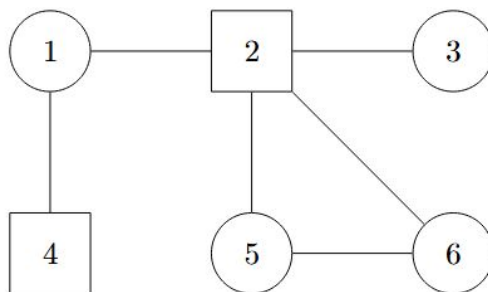
Q2: Bellman-Ford and Floyd-Warshall

- If p is the *unique* shortest s -to- t path in a graph—i.e., every other s -to- t path has strictly larger length—and v is a vertex on p , then the s -to- v portion of p is the *unique* shortest s -to- v path in the graph.
- In the Floyd–Warshall recurrence, the values $d^{[i]}(s, t)$ are *non-increasing* as i increases, i.e., $d^{[j]}(s, t) \leq d^{[i]}(s, t)$ for all $j \geq i$ and all vertices s, t .
- If a graph has no negative-weight cycle, then no shortest path in the graph contains a cycle.
- To fill each individual entry of the DP table in the Floyd–Warshall algorithm, we need to look at only a constant number of cells, whereas this may not be the case in the Bellman–Ford algorithm.

Q3: Dominating sets

A dominating set S in a graph G is a set of vertices for which every vertex of G either is in S , or is adjacent to some vertex in S . We are interested in a smallest dominating set, i.e., one that has the fewest possible vertices. (*There may be more than one.*)

For example, this graph has a smallest dominating set $S^* = \{2, 4\}$: every vertex other than 2 and 4 is adjacent to 2 or 4, and there is no dominating set consisting of a single vertex.



Q3: Dominating sets

```
1: function GREEDYDS( $G$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has at least one vertex do
4:     Select any vertex  $v$  in  $G$  that has largest degree (i.e., the most neighbors)
5:     Add  $v$  to  $S$ 
6:     Remove  $v$  and all its neighbors, including all incident edges, from  $G$ 
7:   return  $S$ 
```

Give a small graph G on which this algorithm might not return a smallest dominating set. Specifically, give a sequence of vertices that the algorithm might choose to make up its final output set, and give an optimal dominating set of G that is smaller than this output set.

Q3: Dominating sets

